

Make folder-`cd(space)/var/www/html`

`Sudo mkdir folder_name`

`cd folder_name`

Open program by `-sudo vim program_name.html`

Run HTML Program

Way 1-On Browser(Mozilla,internet Explorer)on address bar type
`localhost:80/folder_name`

Way 2-Other location/my computer/`var/www/html/folder_name`

1)Design a College/School Registration Form using HTML5 and CSS3
and apply appropriate HTML5 validation attributes.(save file by
.html)

My-Future College / School
Please Fill Below Detail In Student Registration Form.

Student name : Min 3 & Max 10 letter is allow

Date of birth : dd-mm-yyyy Valid Date is allow

Mobile no. : Only integer is allow

Email id : Valid Email Id is allow

Password : Min 8 digit is allow

Gender : ☐ Male ☐ Female

Hobbies : ☐ Drawing ☐ Singing ☐ Dancing ☐ Sketch ☐ Sports

Course : Select Admission Course's

Student photo : No file chosen

Register Time: --:--

Address :

☐ I hereby declare that above information provided is correct

`<!DOCTYPE html>`

`<html>`

`<head>`

`<title>College Registration Form</title>`

`<style>`

```

body { font-family: Arial; background:#f5f5f5; }
.form {
    width: 500px; margin: 30px auto; padding: 20px;
    background: #ffffde5; border: 1px solid #ccc;
}
h2 { text-align:center; }
label { display:inline-block; width:130px; margin:7px 0; }
input, select, textarea { width:300px; padding:5px; }
input[type=radio], input[type=checkbox] { width:auto; }
button { padding:6px 20px; margin:10px; }
</style>
</head>

<body>
<div class="form">
<h2>College Registration Form</h2>

<form method="post" enctype="multipart/form-data">

    <label>Student Name:</label>
    <input type="text" name="name" minlength="3" maxlength="30"
        pattern="[A-Za-z ]+" required><br>

    <label>Date of Birth:</label>
    <input type="date" name="dob" required><br>

    <label>Mobile No.:</label>
    <input type="tel" name="mobile" pattern="[0-9]{10}"
        maxlength="10" required><br>

    <label>Email:</label>
    <input type="email" name="email" required><br>

    <label>Password:</label>
    <input type="password" name="password" minlength="8" required><br>

    <label>Gender:</label>
    <input type="radio" name="gender" value="Male" required> Male
    <input type="radio" name="gender" value="Female"> Female<br>

    <label>Course:</label>
    <select name="course" required>
        <option value="">Select Course</option>

```

```

        <option>BCA</option>
        <option>B.Sc.</option>
        <option>B.Com</option>
        <option>BBA</option>
        <option>MCA</option>
    </select><br>

    <label>Photo:</label>
    <input type="file" name="photo" accept="image/*"><br>

    <label>Address:</label>
    <textarea name="address" rows="3" required></textarea><br>

    <input type="checkbox" required>
    I declare that the above information is correct.<br><br>

    <center>
        <button type="submit">Register</button>
        <button type="reset">Reset</button>
    </center>

</form>
</div>
</body>
</html>

```

2) Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes .(save file by .html)

Student Registration Form

Roll no. :

Student name : -

Father's name :

Date of birth : - - (DD-MM-YYYY)

Mobile no. : -

Email id :

Password :

Gender : ☐ Male ☐ Female

Department : ☐ CSE ☐ IT ☐ ECE ☐ Civil ☐ Mech

Course :

Student photo : No file chosen

City :

Address :

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration Form</title>
  <style>
    body {
      font-family: Arial;
      background: #f8dede;
    }
    .form {
      width: 500px;
      margin: 20px auto;
      padding: 20px;
    }
    h2 { text-align: center; }
    label {
      display: inline-block;
      width: 130px;
      margin: 6px 0;
    }
    input, select, textarea {
      width: 300px;
      padding: 5px;
    }
    input[type="radio"],
    input[type="checkbox"] {
      width: auto;
    }
    button {
      padding: 6px 20px;
      margin-top: 10px;
    }
  </style>
</head>

<body>
<div class="form">
  <h2>Student Registration Form</h2>

  <form method="post" enctype="multipart/form-data">
```

<label>Roll No.:</label>
<input type="text" name="rollno" pattern="[0-9]{1,10}" required>

<label>Student Name:</label>
<input type="text" name="fname" placeholder="First Name"
pattern="[A-Za-z]+" required>
<input type="text" name="lname" placeholder="Last Name"
pattern="[A-Za-z]+" required>

<label>Father's Name:</label>
<input type="text" name="father" required>

<label>Date of Birth:</label>
<input type="date" name="dob" required>

<label>Mobile No.:</label>
<input type="tel" name="mobile" pattern="[6-9][0-9]{9}"
maxlength="10" required>

<label>Email:</label>
<input type="email" name="email" required>

<label>Password:</label>
<input type="password" name="password" minlength="8" required>

<label>Gender:</label>
<input type="radio" name="gender" value="Male" required>Male
<input type="radio" name="gender" value="Female">Female

<label>Department:</label>
<input type="checkbox" name="dept" value="CSE">CSE
<input type="checkbox" name="dept" value="IT">IT
<input type="checkbox" name="dept" value="ECE">ECE

<label>Course:</label>
<select name="course" required>
 <option value="">Select Course</option>
 <option>BCA</option>
 <option>B.Sc.</option>
 <option>B.Com</option>

```
<option>BBA</option>
<option>MCA</option>
</select><br>
```

```
<label>Student Photo:</label>
<input type="file" name="photo" accept="image/*"><br>
```

```
<label>City:</label>
<input type="text" name="city" required><br>
```

```
<label>Address:</label>
<textarea name="address" rows="3" required></textarea><br>
```

```
<center>
  <button type="submit">Register</button>
</center>
```

```
</form>
</div>
</body>
</html>
```

3) Create a responsive webpage using CSS Flexbox for a college homepage containing header, navigation, content and footer. Use appropriate Flexbox alignment and distribution properties .(save file by .html)

```
<!DOCTYPE html>
<html>
<head>
  <title>ABC College</title>
  <style>
    body {
      font-family: Arial;
      margin: 0;
      background: #f4f6f8;
    }
    header, footer {
      background: #123c69;
      color: white;
      padding: 20px;
```

```
    text-align: center;
}
nav {
    background: #0b2545;
    padding: 12px;
    text-align: center;
}
nav a {
    color: white;
    text-decoration: none;
    margin: 15px;
}
.container {
    display: flex;
    gap: 20px;
    margin: 20px auto;
    max-width: 1000px;
}
main, aside {
    background: white;
    padding: 20px;
}
main { flex: 3; }
aside { flex: 1; }
.courses {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}
.course {
    background: #eaf4ff;
    border: 1px solid #b7d7f5;
    padding: 15px;
    text-align: center;
    flex: 1;
}
.notice {
    margin-top: 20px;
    padding: 15px;
    background: #fff3cd;
```

```
    }
    li {
        padding: 7px;
    }
</style>
</head>

<body>

<header>
    <h1>ABC College</h1>
    <p>Knowledge • Excellence • Success</p>
</header>

<nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Courses</a>
    <a href="#">Admission</a>
    <a href="#">Contact</a>
</nav>

<div class="container">

<main>
    <h2>Welcome to ABC College</h2>
    <p>ABC College provides quality education and develops professional
skills.</p>

    <h2>Our Courses</h2>
    <div class="courses">
        <div class="course"><b>BCA</b><br>Computer Applications</div>
        <div class="course"><b>B.Sc.</b><br>Science</div>
        <div class="course"><b>B.Com</b><br>Commerce</div>
        <div class="course"><b>BBA</b><br>Business Administration</div>
    </div>

    <div class="notice">
        <b>Notice:</b> Admissions are now open.
    </div>
```

```
</main>

<aside>
  <h3>Quick Links</h3>
  <ul>
    <li>Student Login</li>
    <li>Online Admission</li>
    <li>Examination</li>
    <li>Time Table</li>
    <li>Results</li>
  </ul>
</aside>

</div>

<footer>
  <p>ABC College, Main Road, Maharashtra</p>
  <p>Email: info@abccollege.edu | Phone: +91 98765 43210</p>
  <p>Facebook | Instagram | Twitter</p>
</footer>

</body>
</html>
```

4) Create a webpage containing a heading and a button. When the button is clicked, change the heading text from “Hello! Welcome” to “Text Changed Successfully!”. (save file by .html)

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Change Heading Text</title>

  <style>
```

```
body {  
    font-family: Arial, sans-serif;  
    text-align: center;  
    margin-top: 100px;  
    background-color: #f2f2f2;  
}
```

```
h1 {  
    color: #333;  
}
```

```
button {  
    padding: 10px 20px;  
    font-size: 16px;  
    background-color: #007bff;  
    color: white;  
    border: none;  
    border-radius: 5px;  
    cursor: pointer;  
}  
  
button:hover {  
    background-color: #0056b3;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 id="heading">Hello! Welcome</h1>
<button onclick="changeText()">Click Me</button>
<script>
    function changeText() {
        document.getElementById("heading").innerHTML =
            "Text Changed Successfully!";
    }
</script>
</body>
</html>
```

5) Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout. [.\(save file by .html\)](#)

```
<!DOCTYPE html>
<html>
<head>
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial;
            margin: 0;
            background: #f4f4f4;
        }

        header {
            text-align: center;
            background: #222;
            color: white;
            padding: 20px;
        }

        .gallery {
            display: grid;
```

```

        grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
        gap: 20px;
        padding: 20px;
    }

    .item {
        background: white;
        text-align: center;
        box-shadow: 0 3px 10px #aaa;
        overflow: hidden;
    }

    .item img {
        width: 100%;
        height: 200px;
        object-fit: cover;
        transition: 0.3s;
    }

    .item:hover img {
        transform: scale(1.1);
        opacity: 0.7;
    }

    .caption {
        padding: 10px;
    }

    @media(max-width:600px) {
        .gallery {
            grid-template-columns: 1fr;
        }
    }
</style>
</head>

<body>

<header>
    <h1>My Image Gallery</h1>

```

```
<p>Responsive HTML5 & CSS3 Gallery</p>
</header>
```

```
<div class="gallery">
```

```
  <div class="item">
    
    <div class="caption">
      <h3>Mountain</h3>
      <p>Beautiful mountain landscape</p>
    </div>
  </div>
```

```
  <div class="item">
    
    <div class="caption">
      <h3>Forest</h3>
      <p>Peaceful forest scenery</p>
    </div>
  </div>
```

```
  <div class="item">
    
    <div class="caption">
      <h3>Nature</h3>
      <p>A beautiful moment in nature</p>
    </div>
  </div>
```

```
  <div class="item">
    
    <div class="caption">
      <h3>Beach</h3>
      <p>Relaxing coastal scenery</p>
    </div>
  </div>
```

```
</div>
```

```
</body>
```

</html>

6) Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class) (save file by .html)

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Hover Effect Example</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      text-align: center;
```

```
      padding-top: 100px;
```

```
      background-color: #f2f2f2;
```

```
    }
```

```
    /* Button styling */
```

```
    button {
```

```
      padding: 12px 25px;
```

```
      font-size: 16px;
```

```
      color: white;
```

```
      background-color: #007bff;
```

```
      border: none;
```

```
      border-radius: 6px;
```

```
      cursor: pointer;
```

```
        transition: 0.3s;
    }
    /* Button hover effect */
    button:hover {
        background-color: #0056b3;
        transform: scale(1.1);
    }

    /* Link styling */
    a {
        display: inline-block;
        margin: 30px;
        font-size: 18px;
        color: #e63946;
        text-decoration: none;
        transition: 0.3s;
    }

    /* Link hover effect */
    a:hover {
        color: #1d3557;
        text-decoration: underline;
        font-size: 20px;
    }
</style>
</head>
```

```
<body>
  <h1>Mouse Hover Effects</h1>
  <button>Hover Over Me</button>
  <br>
  <a href="#">Hover Over This Link</a>
</body>
</html>
```

7) Create an unordered list of Programming Language names and apply different styles to the first and last list items. (Use :first-child and :last-child)
(save file by .html)

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Programming Languages</title>

  <style>

    ul {

      width: 300px;

      font-family: Arial, sans-serif;

    }

    li {

      padding: 10px;

      margin: 5px;

      background-color: #f2f2f2;

    }

  </style>

</head>

<body>

  <div>

    <h1>Programming Languages</h1>

    <ul>

      <li>C</li>

      <li>C++</li>

      <li>C#</li>

      <li>Java</li>

      <li>JavaScript</li>

      <li>Python</li>

      <li>Ruby</li>

      <li>Swift</li>

      <li>TypeScript</li>

      <li>Visual Basic</li>

    </ul>

  </div>

</body>

</html>
```

```
/* Style the first list item */
li:first-child {
    background-color: #007bff;
    color: white;
    font-weight: bold;
}

/* Style the last list item */
li:last-child {
    background-color: #28a745;
    color: white;
    font-style: italic;
}

</style>
</head>
<body>
    <h2>Programming Languages</h2>
    <ul>
        <li>Python</li>
        <li>Java</li>
        <li>C++</li>
        <li>JavaScript</li>
        <li>C#</li>
    </ul>
</body>
</html>
```

8) Create a webpage using JavaScript DOM manipulation and event handling with an image and a button. When the button is clicked, replace the displayed image with another image.(save file by .html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Change Image</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      padding: 50px;
      background-color: #f2f2f2;
    }
    img {
      width: 400px;
      height: 250px;
      object-fit: cover;
      border-radius: 10px;
      margin-bottom: 20px;
    }
    button {
      display: block;
```

```
    margin: auto;
    padding: 12px 25px;
    font-size: 16px;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #0056b3;
}

</style>
</head>
<body>
    <h1>JavaScript Image Change</h1>
    
    <button id="changeButton">Change Image</button>
    <script>
        // Get the image and button elements
        const image = document.getElementById("myImage");
        const button = document.getElementById("changeButton");
        // Add click event to the button
        button.addEventListener("click", function() {
```

```
// Replace the displayed image
image.src = "https://picsum.photos/id/1025/400/250";
image.alt = "Second Image";
});
</script>
</body>
</html>
```

9) Write a JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division. (SAVE-sudo vim pg_name.js RUN-node pg_name.js)

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
const divide = (a, b) => a / b;
let a = 20;
let b = 5;
console.log("Addition: " + add(a, b));
console.log("Subtraction: " + subtract(a, b));
console.log("Multiplication: " + multiply(a, b));
console.log("Division: " + divide(a, b));
```

10) Write a JavaScript program to store student information and marks in variables and generate a formatted student report using template literals
save (sudo vim sl10.js) (run : node sl10.js)

```
// Student information
let name = "Rahul Sharma";
let rollNo = 101;
let age = 18;
```

```

// Marks

let math = 85;

let science = 90;

let english = 78;


// Calculate total and percentage

let total = math + science + english;

let percentage = total / 3;


// Generate formatted student report using template literal

let report = `

=====

      STUDENT REPORT

=====

Name      : ${name}

Roll No.   : ${rollNo}

Age        : ${age}

Marks:

Math       : ${math}

Science    : ${science}

English    : ${english}

Total Marks: ${total}

Percentage : ${percentage.toFixed(2)}%

=====

`;

```

```
// Display the report
```

```
console.log(report);
```

11) Write a JavaScript program using an arrow function to calculate the total and average marks of three subjects. Display the result using template literals.

save (sudo vim sl11.js) (run : node sl11.js)

```
const calculateMarks = (math, science, english) => {
```

```
    let total = math + science + english;
```

```
    let average = total / 3;
```

```
    return { total, average };
```

```
};
```

```
// Marks of three subjects
```

```
let math = 85;
```

```
let science = 90;
```

```
let english = 78;
```

```
// Call the arrow function
```

```
let result = calculateMarks(math, science, english);
```

```
// Display the result using template literals
```

```
console.log(`
```

```
Student Marks Report
```

```
-----
```

```
Math    : ${math}
```

```
Science : ${science}
```

```
English : ${english}
```

```
Total   : ${result.total}
```

```
Average : ${result.average.toFixed(2)}
```

```
`);
```

12) Write a JavaScript program using an arrow function to calculate the total price of a product. Display the bill using template literals and string interpolation. save (sudo vim sl12.js) (run : node sl12.js)

```
// Arrow function to calculate total price
const calculateTotal = (price, quantity) => price * quantity;

// Product details
let productName = "Laptop";
let price = 50000;
let quantity = 2;

// Calculate total
let total = calculateTotal(price, quantity);

// Generate bill using template literals
let bill = `
=====

      PRODUCT BILL
=====

Product : ${productName}
Price   : ₹${price}
Quantity : ${quantity}
-----

Total   : ₹${total}
=====

`;

// Display the bill
console.log(bill);
```

13) Write a JavaScript program to swap two numbers without using a third variable using array destructuring save (sudo vim sl13.js) (run : node sl13.js)

```
let a = 10;
let b = 20;
console.log(Before swapping: a = ${a}, b = ${b});
// Swap using array destructuring
[a, b] = [b, a];
console.log(After swapping: a = ${a}, b = ${b});
```

14) Create an object containing employee details such as name, department, and salary. Use object destructuring to extract and display the values.

save (sudo vim sl14.js) (run : node sl14.js)

```
let employee = {
  name: "Rahul Sharma",
  department: "IT",
  salary: 50000
};
// Object destructuring
let { name, department, salary } = employee;
// Display employee details
console.log(`
Employee Details
-----

Name    : ${name}
Department : ${department}
Salary   : ₹${salary}
`);
```

```

let employee = {
    name: "Rahul Sharma",
    department: "IT",
    salary: 50000
};

// Object destructuring
let { name, department, salary } = employee;

// Display employee details
console.log(`
Employee Details
-----
Name    : ${name}
Department : ${department}
Salary   : ₹${salary}
`);

```

15) Write a JavaScript program to create a copy of an existing array using the spread operator and add a new element to the copied array.

save (sudo vim sl15.js) (run : node sl15.js)

```

let originalArray = ["Apple", "Banana", "Mango"];

// Create a copy using the spread operator
let copiedArray = [...originalArray];

// Add a new element to the copied array
copiedArray.push("Orange");

// Display both arrays
console.log(Original Array: ${originalArray});
console.log(Copied Array: ${copiedArray});

```

16) Write a JavaScript program using a rest parameter to accept any number of values and calculate their sum save (sudo vim sl16.js) (run : node sl16.js)

```
function calculateSum(...numbers) {  
    let sum = 0;  
    for (let number of numbers) {  
        sum += number;  
    }  
    return sum;  
}  
  
// Pass any number of values  
let result = calculateSum(10, 20, 30, 40, 50);  
  
// Display the result  
console.log(The sum of the numbers is: ${result});
```

18) Create a Node.js HTTP server that sends an appropriate response to a client request. save (sudo vim sl18.js) (run : node sl18.js)

```
const http = require("http");  
  
const server = http.createServer((req, res) => {  
    res.writeHead(200, { "Content-Type": "text/html" });  
  
    res.end(`  
        <h1>Hello from Node.js!</h1>  
        <p>Server received your request successfully.</p>  
    `);  
});  
  
server.listen(3000, () => {
```

```
    console.log("Server is running at http://localhost:3000");  
  });
```

20) Write a JavaScript program using a Promise to divide two numbers. If the denominator is zero, reject the Promise and handle the error.

save (sudo vim sl20.js) (run : node sl20.js)

```
function divide(a, b) {  
  return new Promise((resolve, reject) => {  
    if (b === 0) {  
      reject("Error: Cannot divide by zero!");  
    } else {  
      resolve(a / b);  
    }  
  });  
}
```

// Example

```
let num1 = 20;
```

```
let num2 = 5;
```

```
divide(num1, num2)  
  .then(result => {  
    console.log(Result: ${result});  
  })  
  .catch(error => {  
    console.log(error);  
  });
```

21)Write a Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

save (sudo vim sl21.js) (run : node sl21.js)

```
const path = require("path");
// File and directory paths
let directory = "documents";
let file = "report.txt";
// 1. Join paths
let joinedPath = path.join(directory, "2026", file);
// 2. Resolve path to an absolute path
let absolutePath = path.resolve(joinedPath);
// 3. Extract file/directory information
let fileName = path.basename(joinedPath);
let directoryName = path.dirname(joinedPath);
let extension = path.extname(joinedPath);
// Display results
console.log(`Path Operations
=====
Joined Path   : ${joinedPath}
Absolute Path : ${absolutePath}
File Name    : ${fileName}
Directory Name : ${directoryName}
File Extension : ${extension}
`);
```